



Week 6 Workshop

Language Models

Retrieving Relevant Documents using Language Models

IFN647

Tutor

School of Computer Science
Queensland University of Technology

Learning Objectives

- Understand inverted index construction (covered in last week's workshop)
- Apply language model scoring
- Implement Jelinek-Mercer smoothing

Tasks

➤ Task 1:

Build an inverted index from a folder of documents using `index_docs()`. (covered in last week's workshop)

➤ Task 2:

Compute document relevance scores for a query using maximum likelihood estimation. (Reference program available in the lecture notes)

➤ Task 3:

Apply Jelinek-Mercer smoothing to improve retrieval with collection-level statistics.

➤ Task 4:

Test and compare your retrieval functions using provided document sets and queries.

Knowledge Hints: Language Models

A language model is a **probability distribution over words** that tells us how likely a word is to appear in a given context.

- Example: If a document collection contains only 5 words:

`{w1, w2, w3, w4, w5}`, a language model might assign:

`P(w1) = 0.2, P(w2) = 0.1, P(w3) = 0.35, P(w4) = 0.25, P(w5) = 0.1`

- It can be used to **generate sentences** or measure relevance between documents and queries.

Language Models in Information Retrieval (IR)

- In IR, language models estimate how likely is this document to “generate” a query.
 - Example:
Query: `{econom, espionag}`
Document A: `{US, econom, spy}`
→ Compute: $P(Q|D_A) = P(\text{econom} | A) \times P(\text{espionag} | A)$
 - This probability helps us **rank documents based on relevance**.
- **Raw likelihood** (Query-likelihood), **Smoothed likelihood** with **Jelinek-Mercer** (JM Smoothing)

Knowledge Hints: Query-Likelihood Model

- In IR, language models estimate **how likely a document is to generate a query**.
- This approach is known as the **Query Likelihood Model**.

$$P(Q|D) = \prod_{i=1}^n P(q_i|D) \quad \text{where,}$$

$$P(q_i|D) = \frac{f_{q_i,D}}{|D|}$$

- $f_{q_i,D}$: frequency of term q_i in document D
- $|D|$: total number of terms in document D

Query: $Q = \{\text{econom, spy}\}$

Document A: $D = \{\text{US, econom, spy}\}$

- $|D| = 3$, $f(\text{"econom"}, D) = 1$, $f(\text{"spy"}, D) = 1$
- $P(Q|D) = \frac{1}{3} \times \frac{1}{3} = \frac{1}{9} = 0.111$

Query: $Q = \{\text{econom, spy, espionag, market}\}$

Document B: $D = \{\text{econom, spy, espionag}\}$

- $|D| = 3$, $f(\text{"market"}, D) = 0$
- $P(Q|D) = \frac{1}{3} \times \frac{1}{3} \times \frac{1}{3} \times 0 = 0$

Query: $Q = \{\text{econom, spy, espionag, market}\}$

Document C: $D = \{\text{econom}\}$

- $|D| = 1$, $f(\text{"spy"}, D) = 0$, $f(\text{"espionag"}, D) = 0$, $f(\text{"market"}, D) = 0$
- $P(Q|D) = \frac{1}{1} \times 0 \times 0 \times 0 = 0$

- If query words are missing from document, score will be zero
 - This is the major problem.
 - E.g., missing 1 out of 4 query words same as missing 3 out of 4.
 - This is clearly not appropriate for longer queries.

- Longer queries are **more likely** to contain at least one term that doesn't appear in a relevant document.
- As the query length increases, the **chance of scoring a document as 0 grows**, even if the document matches most of the query well.

Knowledge Hints: Jelinek-Mercer Smoothing

◆ Why Smoothing?

- In the query likelihood method, missing query terms make $P(Q|D) = 0$, even if the document is mostly relevant.
- Smoothing assigns some probability to unseen terms by based on the overall collection.

$$P(Q|D) = \prod_{i=1}^n P(q_i|D) \quad \text{where,} \quad p(q_i|D) = (1 - \lambda) \frac{f_{q_i,D}}{|D|} + \lambda \frac{c_{q_i}}{|C|}$$

- $f_{q_i,D}$: frequency of term q_i in document D
- c_{q_i} : frequency of term q_i in the collection C
- $|D|$: total number of terms in D
- $|C|$: total number of terms in the collection C
- λ : smoothing parameter (e.g., 0.4)

◆ Example (Using $\lambda = 0.5$)

Query: $Q = \{\text{US, econom, espionag}\}$

Document D : $\{\text{US, econom, spy}\}$

Collection C : $\{\text{D1, D2, D3, D4, D5, D6}\}$ $|C| = 25, c_{US} = 4, c_{econom} = 3, c_{espionag} = 3$

- $P(US|D) = 0.5 \cdot \frac{1}{3} + 0.5 \cdot \frac{4}{25} = 0.166 + 0.08 = 0.246$
- $P(econom|D) = 0.5 \cdot \frac{1}{3} + 0.5 \cdot \frac{3}{25} = 0.166 + 0.06 = 0.226$
- $P(espionag|D) = 0.5 \cdot 0 + 0.5 \cdot \frac{3}{25} = 0 + 0.06 = 0.06$
- $P(Q|D) = 0.246 \times 0.226 \times 0.06 = 0.0033$

✓ Benefit:

- The result is non-zero even when some query terms are missing from the document.
- This allows for **fairer document ranking** in real-world search systems.

Task1

covered in last week's workshop

Task 1. Design function `index_docs()` to construct an inverted index, a dictionary

```
{term:{docID1:freq1, DocID2:freq2, ...}, ...}.
```

For each document in the "Test_Doc" folder, it firstly finds the "docid" in tag <newsitem>. For each index term (excluding number, punctuations, and tags <p> and </p> in <text>), it aslo inserts the term into the index or accumulates its frequency.

Workshop Task 1-Solution...

From previous week...

Task 1. For the given two XML documents (you can download them from week 3 workshop and then save them in a folder, e.g., 'data'), design a python function `index_docs()` to index them (please remove stop words and index stems only). The returned index should be a dictionary {term:{docID1:freq1, DocID2:freq2}, ...}

The following are a sample of outcomes, where '6146' and '741299' are the document id, respectively.

```
Index = {'argentin': {'6146': 1}, 'bond': {'6146': 1},
'slight': {'6146': 2}, 'higher': {'6146': 1}, 'small':
{'6146': 1}, 'technic': {'6146': 2}, 'bounc': {'6146': 2},
'wednesday': {'6146': 1}, ..., 'time': {'741299': 1}, 'abl':
{'741299': 1}, 'push': {'741299': 1}, 'hard': {'741299':
1}, 'big': {'741299': 1}, 'third': {'741299': 1}, 'finish':
{'741299': 1}, 'porsch': {'741299': 1}, 'franc': {'741299':
1}, 'bob': {'741299': 1}, 'wollek': {'741299': 1},
'yannick': {'741299': 1}, 'dalma': {'741299': 1},
'belgian': {'741299': 2}, 'thierri': {'741299': 1},
'boutsen': {'741299': 1}, 'former': {'741299': 1},
'formula': {'741299': 1}, 'one': {'741299': 1}, 'driver':
{'741299': 1}, 'switch': {'741299': 1}, 'normal':
{'741299': 1}, 'share': {'741299': 1}, 'han': {'741299':
1}, 'stuck': {'741299': 1}, 'follow': {'741299': 1},
'power': {'741299': 1}, 'steer': {'741299': 1}, 'failur':
{'741299': 1}}
```

Workshop Task 1-Solution...

```
import glob, os
import string
from stemming.porter2 import stem
```



```
def index_docs(inputpath, stop_words):
```

```
    Index = {} # initialize the index
```

```
    os.chdir(inputpath)
```

```
    for file_ in glob.glob("*.xml"):
```

```
        start_end = False
```

```
        for line in open(file_):
```

```
            line = line.strip()
```

```
            if (start_end == False):
```

```
                if line.startswith("<newsitem "):
```

```
                    for part in line.split():
```

```
                        if part.startswith("itemid="):
```

```
                            docid = part.split("=")[1].split("\\"")[1]
```

```
                            break
```

```
                if line.startswith("<text>"):
```

```
                    start_end = True
```

```
            elif line.startswith("</text>"):
```

```
                break
```

```
        else:
```

```
            line = line.replace("<p>", "").replace("</p>", "")
```

```
            line = line.translate(str.maketrans('', '', string.digits)).translate(str.maketrans(string.punctuation, ' '*len(string.punctuation)))
```

```
            for term in line.split():
```

```
                term = stem(term.lower())
```

```
                if len(term) > 2 and term not in stop_words:
```

```
                    try:
```

```
                        try:
```

```
                            Index[term][docid] += 1
```

```
                        except KeyError:
```

```
                            Index[term][docid]=1
```

```
                    except KeyError:
```

```
                        Index[term] = {docid:1}
```

```
    return Index
```



This is our inverted index dictionary



Loop through all .xml files



Reads one line of each file at a time

From previous week...

`glob.glob("*.xml")` returns a list of filenames (strings) that match the pattern *.xml.

```
import glob
for name in glob.glob('*'):
    print(name)
```

```
6146.xml
741299newsML.xml
common-english-words.txt
example.html
seq1.txt
seq3.txt
Test4.ipynb
Test.ipynb
Test5.ipynb
test_wk2.ipynb
week02_solution.ipynb
wk3_solutions.ipynb
wk4_solutions.ipynb
wk5_solutions.ipynb
workshop_w2_test.ipynb
```

```
import glob
for name in glob.glob('*.xml'):
    print(name)
```

```
6146.xml
741299newsML.xml
```



{term: {docID1:freq1, DocID2:freq2}, ...}

If term & docid already exist → increment frequency

If term exists but docid is new → add docid

If term is new → create new entry in the index

Workshop Task 1 - Solution Execution (Jupyter notebook / Cmd)...

From previous week...

```
#We assume the Jupyter notebook file starts from your working directory.

curr_path=os.getcwd() # We use os methods to find the current working directory 'curr_path'
print(curr_path)

stopwords_f = open('common-english-words.txt', 'r')
stop_words = stopwords_f.read().split(',')
stopwords_f.close()

data_path = curr_path+'/data' # to find data's directory 'data_path'.

Index = index_docs(data_path, stop_words) #create an index for all terms in <text>, data structure {'w1':{'ID1':2, 'ID2':1}, 'w2
os.chdir(curr_path) # go back to the current working directory after call index_docs as it changed the working directory.

print(Index)
```

Task 2

Task 2: Let $Q = \{q_1:1, q_2:1, \dots, q_n:1\}$ be a dictionary, please define a function `likelihood_IR(I, Q)` to estimate $P(Q|D)$, i.e., it returns the score of document D for the given query Q by using

$$P(Q|D) = \prod_{i=1}^n P(q_i|D)$$

$$P(q_i|D) = \frac{f_{q_i,D}}{|D|}$$

where $f_{q_i,D}$ is the number of times word q_i occurs in document D , and $|D|$ is the number of words in D .

`I`

`Index = index_docs(data_path, stop_words)`

Input:

Go to Task4 to know about inputs:

Test the defined functions using the provided set of test documents (in folder "Test_docs").

You need to use the 'common-Englishwords.txt' when indexing both sets of documents into Index_Test.

you can assume `lamda = 0.4`, and `Query = {'formula':1, 'one':1}, {'share':1}` or `{ 'leaderboard':1, 'british':1}`.

`likelihood_IR(I, Q)`

Output for Query2, {'share':1}

Likelihood Query Result -----

Document ID: 807606 and relevance weight: 0.017937219730941704

Document ID: 809481 and relevance weight: 0.011299435028248588

Document ID: 807600 and relevance weight: 0.003179650238473768

Task 2: Solution

$I = \{\text{term}: \{\text{docID1:freq1}, \text{DocID2:freq2}\}, \dots\}$
 $Q = \{\text{'formula':1}, \text{'one':1}\}$

```
def likelihood_IR(I, Q): # index I is a Dictionary of term:Dictionary of (itemId:freq)
    L={}                # Holds inverted lists of query terms
    R={}                # Document scores: {docID: P(Q|D)}
    D_len={}            # Document lengths: {docID: total terms}
```

Step 1: Define variables

$$P(Q|D) = \prod_{i=1}^n P(q_i|D)$$

$$P(q_i|D) = \frac{f_{q_i,D}}{|D|}$$

Task 2: Solution

$I = \{\text{term}:\{\text{docID1:freq1, DocID2:freq2}\}, \dots\}$
 $Q = \{\text{'formula':1, 'one':1}\}$

```
def likelihood_IR(I, Q): # index I is a Dictionary of term:Dictionary of (itemId:freq)
    L={}                # Holds inverted lists of query terms
    R={}                # Document scores: {docID: P(Q|D)}
    D_len={}            # Document lengths: {docID: total terms}
```

Step 1: Define variables

```
    for list in I.items():
        for id in list[1].items():
            R[id[0]]=1    # R = {id1:1, id2:1, id3:1, ...}
            D_len[id[0]]=0.5 # D_len = {id1:0.5, id2:0.5, id3:0.5, ...}
```

Step 2: Initialize variables

$$P(Q|D) = \prod_{i=1}^n P(q_i|D)$$

$$P(q_i|D) = \frac{f_{q_i,D}}{|D|}$$

Task 2: Solution

$I = \{\text{term}:\{\text{docID1:freq1, DocID2:freq2}\}, \dots\}$

$Q = \{\text{'formula':1, 'one':1}\}$

```
def likelihood_IR(I, Q): # index I is a Dictionary of term:Dictionary of (itemId:freq)
    L={}                # Holds inverted lists of query terms
    R={}                # Document scores: {docID: P(Q|D)}
    D_len={}           # Document lengths: {docID: total terms}
```

Step 1: Define variables

```
    for list in I.items():
        for id in list[1].items():
            R[id[0]]=1    # R = {id1:1, id2:1, id3:1, ...}
            D_len[id[0]]=0.5 # D_len = {id1:0.5, id2:0.5, id3:0.5, ...}
```

Step 2: Initialize variables

```
    if (list[0] in Q):
        L[list[0]] = I[list[0]] # L = {'formula':{id1:freq1, id2:freq2}, 'one': {id1:freq3}}
```

Step 3: Filter inverted index from I only for Q term

$$P(Q|D) = \prod_{i=1}^n P(q_i|D)$$

$$P(q_i|D) = \frac{f_{q_i,D}}{|D|}$$

Task 2: Solution

$I = \{\text{term}:\{\text{docID1:freq1, DocID2:freq2}\}, \dots\}$

$Q = \{\text{'formula':1, 'one':1}\}$

```
def likelihood_IR(I, Q): # index I is a Dictionary of term:Dictionary of (itemId:freq)
    L={}                # Holds inverted lists of query terms
    R={}                # Document scores: {docID: P(Q|D)}
    D_len={}           # Document lengths: {docID: total terms}
```

Step 1: Define variables

```
    for list in I.items():
        for id in list[1].items():
            R[id[0]]=1    # R = {id1:1, id2:1, id3:1, ...}
            D_len[id[0]]=0.5 # D_len = {id1:0.5, id2:0.5, id3:0.5, ...}
```

Step 2: Initialize variables

```
    if (list[0] in Q):
        L[list[0]] = I[list[0]] # L = {'formula':{id1:freq1, id2:freq2}, 'one': {id1:freq3}}
```

Step 3: Filter inverted index from I only for Q term

Consider this $Q = \{\text{'formula':1, 'one':1, 'right':2}\}$ where 'right' is not in I

$$P(Q|D) = \prod_{i=1}^n P(q_i|D)$$

$$P(q_i|D) = \frac{f_{q_i,D}}{|D|}$$

Task 2: Solution

$I = \{\text{term:}\{\text{docID1:freq1, DocID2:freq2}\}, \dots\}$

$Q = \{\text{'formula':1, 'one':1}\}$

```
def likelihood_IR(I, Q): # index I is a Dictionary of term:Dictionary of (itemId:freq)
    L={}                # Holds inverted lists of query terms
    R={}                # Document scores: {docID: P(Q|D)}
    D_len={}            # Document lengths: {docID: total terms}
```

Step 1: Define variables

```
    for list in I.items():
        for id in list[1].items():
            R[id[0]]=1    # R = {id1:1, id2:1, id3:1, ...}
            D_len[id[0]]=0.5 # D_len = {id1:0.5, id2:0.5, id3:0.5, ...}
```

Step 2: Initialize variables

```
    if (list[0] in Q):
        L[list[0]] = I[list[0]] # L = {'formula':{id1:freq1, id2:freq2}, 'one': {id1:freq3}}
```

Step 3: Filter inverted index from I only for Q term

```
    for q_term in Q.items(): # L may not include all query terms
        if not(q_term[0] in L):
            L[q_term[0]]={} # L = {'formula':{id1:freq1, id2:freq2}, 'one': {id1:freq3}, 'right': {}}
```

Consider this $Q = \{\text{'formula':1, 'one':1, 'right':2}\}$ where 'right' is not in I

$$P(Q|D) = \prod_{i=1}^n P(q_i|D)$$

$$P(q_i|D) = \frac{f_{q_i,D}}{|D|}$$

Task 2: Solution

$I = \{\text{term}:\{\text{docID1:freq1, DocID2:freq2}\}, \dots\}$

$Q = \{\text{'formula':1, 'one':1}\}$

```
def likelihood_IR(I, Q): # index I is a Dictionary of term:Dictionary of (itemID:freq)
    L={}                 # Holds inverted lists of query terms
    R={}                 # Document scores: {docID: P(Q|D)}
    D_len={}            # Document lengths: {docID: total terms}
```

Step 1: Define variables

```
    for list in I.items():
        for id in list[1].items():
            R[id[0]]=1      # R = {id1:1, id2:1, id3:1, ...}
            D_len[id[0]]=0.5 # D_len = {id1:0.5, id2:0.5, id3:0.5, ...}
```

Step 2: Initialize variables

```
    if (list[0] in Q):
        L[list[0]] = I[list[0]] # L = {'formula':{id1:freq1, id2:freq2}, 'one':{id1:freq3}}
```

Step 3: Filter inverted index from I only for Q term

```
    for q_term in Q.items(): # L may not include all query terms
        if not(q_term[0] in L):
            L[q_term[0]]={} # L = {'formula':{id1:freq1, id2:freq2}, 'one':{id1:freq3}, 'right':{}}
```

Consider this $Q = \{\text{'formula':1, 'one':1, 'right':2}\}$ where 'right' is not in I

```
    for list in I.items():
        for id in list[1].items(): # Count term occurrences in documents
            D_len[id[0]] = D_len[id[0]] + id[1] # D_len = {id1:2.5, id2:4.5, id3:7.5, ...}
```

Step 4: Calculate actual document lengths (total number of terms in the doc)

$$P(Q|D) = \prod_{i=1}^n P(q_i|D)$$

$$P(q_i|D) = \frac{f_{q_i,D}}{|D|}$$

Task 2: Solution

$I = \{\text{term:}\{\text{docID1:freq1, DocID2:freq2}\}, \dots\}$

$Q = \{\text{'formula':1, 'one':1}\}$

```
def likelihood_IR(I, Q): # index I is a Dictionary of term:Dictionary of (itemID:freq)
    L={}                # Holds inverted lists of query terms
    R={}                # Document scores: {docID: P(Q|D)}
    D_len={}           # Document lengths: {docID: total terms}
```

Step 1: Define variables

```
    for list in I.items():
        for id in list[1].items():
            R[id[0]]=1      # R = {id1:1, id2:1, id3:1, ...}
            D_len[id[0]]=0.5 # D_len = {id1:0.5, id2:0.5, id3:0.5, ...}
```

Step 2: Initialize variables

```
    if (list[0] in Q):
        L[list[0]] = I[list[0]] # L = {'formula':{id1:freq1, id2:freq2}, 'one':{id1:freq3}}
```

Step 3: Filter inverted index from I only for Q term

```
    for q_term in Q.items(): # L may not include all query terms
        if not(q_term[0] in L):
            L[q_term[0]]={} # L = {'formula':{id1:freq1, id2:freq2}, 'one':{id1:freq3}, 'right':{}}
```

Consider this $Q = \{\text{'formula':1, 'one':1, 'right':2}\}$ where 'right' is not in I

```
    for list in I.items():
        for id in list[1].items(): # Count term occurrences in documents
            D_len[id[0]] = D_len[id[0]] + id[1] # D_len = {id1:2.5, id2:4.5, id3:7.5, ...}
```

Step 4: Calculate actual document lengths (total number of terms in the doc)

```
    for (d, sd) in R.items():
        for (term, f) in L.items():
```

Step 5: Score each document

```
            sd = sd*(f[d]/D_len[d])
        R[d] = sd
    return R
```

$$P(Q|D) = \prod_{i=1}^n P(q_i|D)$$

$$P(q_i|D) = \frac{f_{q_i,D}}{|D|}$$

Task 2: Solution

$I = \{\text{term:}\{\text{docID1:freq1, DocID2:freq2}\}, \dots\}$

$Q = \{\text{'formula':1, 'one':1}\}$

```
def likelihood_IR(I, Q): # index I is a Dictionary of term:Dictionary of (itemID:freq)
    L={}                # Holds inverted lists of query terms
    R={}                # Document scores: {docID: P(Q|D)}
    D_len={}            # Document lengths: {docID: total terms}
```

Step 1: Define variables

```
    for list in I.items():
        for id in list[1].items():
            R[id[0]]=1      # R = {id1:1, id2:1, id3:1, ...}
            D_len[id[0]]=0.5 # D_len = {id1:0.5, id2:0.5, id3:0.5, ...}
```

Step 2: Initialize variables

```
    if (list[0] in Q):
        L[list[0]] = I[list[0]] # L = {'formula':{id1:freq1, id2:freq2}, 'one':{id1:freq3}}
```

Step 3: Filter inverted index from I only for Q term

```
    for q_term in Q.items(): # L may not include all query terms
        if not(q_term[0] in L):
            L[q_term[0]]={} # L = {'formula':{id1:freq1, id2:freq2}, 'one':{id1:freq3}, 'right':{}}
```

Consider this $Q = \{\text{'formula':1, 'one':1, 'right':2}\}$ where 'right' is not in I

```
    for list in I.items():
        for id in list[1].items(): # Count term occurrences in documents
            D_len[id[0]] = D_len[id[0]] + id[1] # D_len = {id1:2.5, id2:4.5, id3:7.5, ...}
```

Step 4: Calculate actual document lengths (total number of terms in the doc)

```
    for (d, sd) in R.items():
        for (term, f) in L.items():
            if not(d in f):
                f[d]=0
            sd = sd*(f[d]/D_len[d])
        R[d] = sd
    return R
```

Step 5: Score each document

$$P(Q|D) = \prod_{i=1}^n P(q_i|D)$$

$$P(q_i|D) = \frac{f_{q_i,D}}{|D|}$$

WHY? # D_len = {id1:0.5, id2:0.5, id3:0.5, ...}

Document A (short)
[machine | learning]
Length = 2

Document B (long)
[machine ... learning ... many words ...]
Length = 100

Without Smoothing

Document A:

$$P(\text{machine} | A) = 1/2 = 0.5$$

$$P(\text{learning} | A) = 1/2 = 0.5$$

Final:

$$0.5 \times 0.5 = \mathbf{0.25}$$

Document B:

$$P(\text{machine} | B) = 5/100 = 0.05$$

$$P(\text{learning} | B) = 5/100 = 0.05$$

Final:

$$0.05 \times 0.05 = \mathbf{0.0025}$$

Problem

Document A ranks MUCH higher

But:

- It's just 2 words
- Not informative at all

This is unrealistic ranking

With Smoothing

Document A:

$$\text{Length} = 2 + 0.5 = 2.5$$

$$P(\text{machine} | A) = 1 / 2.5 = 0.4$$

$$P(\text{learning} | A) = 1 / 2.5 = 0.4$$

Final:

$$0.4 \times 0.4 = \mathbf{0.16}$$

Document B:

$$\text{Length} = 100 + 0.5 = 100.5$$

$$P(\text{machine} | B) \approx 5 / 100.5 \approx 0.0497$$

$$\text{Final} \approx \mathbf{0.00247}$$

Short doc → slightly reduced score

Long doc → almost same score

It makes the system:

- less biased
- more realistic

Task 3

Task 3: Assume you have obtained another folder, e.g., "Collection_C". Extend function `likelihood_IR(I, Q)` to a Jelinek-Mercer smoothing function `likelihood_JM(I_C, I_D, Q, lambda)` by using the following equation:

$$P(Q|D) = \prod_{i=1}^n \left((1 - \lambda) \frac{f_{q_i, D}}{|D|} + \lambda \frac{c_{q_i}}{|C|} \right)$$

where c_{q_i} is the number of times query term q_i occurs in I_C , and $|C|$ is the total number of term occurrences in I_C . See more details in page 12 of week 5 lecture notes.

The function estimates a score for documents in I_D .

`I_D`

`Index = index_docs(data_path, stop_words)`

`I_C`

`Index = index_docs(data_path, stop_words)`

`likelihood_JM(I_C, I_D, Q, lambda)`

Input:

Go to Task4 to know about inputs:

Test the defined functions using the provided set of test documents (in folder "Test_docs") and a collection of documents (in folder "collection_C").

You need to use the 'common-Englishwords.txt' when indexing both sets of documents into Index_Test.

you can assume `lambda = 0.4`, and `Query = {'formula':1, 'one':1}, {'share':1}` or `{ 'leaderboard':1, 'british':1}`.

Output for Query2, {'share':1}

```
JM Smoothing-Based Query Result -----
Document ID: 807606 and relevance weight: 0.012240164350880293
Document ID: 809481 and relevance weight: 0.008257493529264423
Document ID: 807600 and relevance weight: 0.003385622655399532
Document ID: 809495 and relevance weight: 0.0014778325123152713
```

Task 3: Solution

$I_C = \{\text{term}: \{\text{docID1:freq1}, \text{DocID2:freq2}, \dots\}\}$

$$P(Q|D) = \prod_{i=1}^n \left((1-\lambda) \frac{f_{q_i, D}}{|D|} + \lambda \frac{c_{q_i}}{|C|} \right)$$

where c_{q_i} is the number of times query term q_i occurs in I_C , and $|C|$ is the total number of term occurrences in I_C .

```
def likelihood_IR(I, Q): # index I is a Dictionary of term:Dictionary of (itemId:freq)
    L={}                # Holds inverted lists of query terms
    R={}                # Document scores: {docID: P(Q|D)}
    D_len={}            # Document lengths: {docID: total terms}

    for list in I.items():
        for id in list[1].items():
            R[id[0]]=1    # R = {id1:1, id2:1, id3:1, ...}
            D_len[id[0]]=0.5

        if (list[0] in Q):
            L[list[0]] = I[list[0]]

    for q_term in Q.items(): # L may not include all query terms
        if not(q_term[0] in L):
            L[q_term[0]]={}    # L = {'formula':{id1:freq1, id2:freq2}, 'one': {id1:freq3}, 'right': { }}

    for list in I.items():
        for id in list[1].items(): # Count term occurrences in documents
            D_len[id[0]] = D_len[id[0]] + id[1]    # D_len = {id1:2.5, id2:4.5, id3:7.5, ...}
```

Task 3: Solution

`likelihood_JM(I_C, I_D, Q, lambda)`

`I_C = {term: {docID1:freq1, DocID2:freq2}, ...}`

```
def likelihood_IR(I, Q): # index I is a Dictionary of term:Dictionary of (itemID:freq)
```

```
    L={} # Holds inverted lists of query terms
```

```
    R={} # Document scores: {docID: P(Q|D)}
```

```
    D_len={} # Document lengths: {docID: total terms}
```

```
    for list in I.items():
```

```
        for id in list[1].items():
```

```
            R[id[0]]=1 # R = {id1:1, id2:1, id3:1, ...}
```

```
            D_len[id[0]]=0.5
```

```
        if (list[0] in Q):
```

```
            L[list[0]] = I[list[0]]
```

```
    for q_term in Q.items(): # L may not include all query terms
```

```
        if not(q_term[0] in L):
```

```
            L[q_term[0]]={} # L = {'formula':{id1:freq1, id2:freq2}, 'one': {id1:freq3}, 'right': { }}
```

```
    for list in I.items():
```

```
        for id in list[1].items(): # Count term occurrences in documents
```

```
            D_len[id[0]] = D_len[id[0]] + id[1] # D_len = {id1:2.5, id2:4.5, id3:7.5, ...}
```

$$P(Q|D) = \prod_{i=1}^n \left((1-\lambda) \frac{f_{q_i, D}}{|D|} + \lambda \frac{c_{q_i}}{|C|} \right)$$

where c_{q_i} is the number of times query term q_i occurs in I_C , and $|C|$ is the total number of term occurrences in I_C .

Task 3: Solution

$I_C = \{\text{term}: \{\text{docID1:freq1}, \text{DocID2:freq2}\}, \dots\}$

```
def likelihood_JM(I_C, I_D, Q, lamda): # index I_C and I_D have the form of {term:{itemId:freq}}
    L={} # Holds inverted lists of query terms
    R={} # Document scores: {docID: P(Q|D)}
    D_len={} # Document lengths: {docID: total terms}

    for list in I.items():
        for id in list[1].items():
            R[id[0]]=1 # R = {id1:1, id2:1, id3:1, ...}
            D_len[id[0]]=0.5

    if (list[0] in Q):
        L[list[0]]= I[list[0]]

    for q_term in Q.items(): # L may not include all query terms
        if not(q_term[0] in L):
            L[q_term[0]]={} # L = {'formula':{id1:freq1, id2:freq2}, 'one': {id1:freq3}, 'right': { }}

    for list in I_D.items():
        for id in list[1].items(): # Count term occurrences in documents
            D_len[id[0]]= D_len[id[0]] + id[1] # D_len = {id1:2.5, id2:4.5, id3:7.5, ...}
```

$$P(Q|D) = \prod_{i=1}^n ((1-\lambda) \frac{f_{q_i,D}}{|D|} + \lambda \frac{c_{q_i}}{|C|})$$

where c_{q_i} is the number of times query term q_i occurs in I_C , and $|C|$ is the total number of term occurrences in I_C .

Task 3: Solution

$I_C = \{\text{term}: \{\text{docID1:freq1}, \text{DocID2:freq2}\}, \dots\}$

```
def likelihood_JM(I_C, I_D, Q, lamda): # index I_C and I_D have the form of {term:{itemId:freq}}
    L={} # Holds inverted lists of query terms
    R={} # Document scores: {docID: P(Q|D)}
    D_len={} # Document lengths: {docID: total terms}

    for list in I.items():
        for id in list[1].items():
            R[id[0]]=1 # R = {id1:1, id2:1, id3:1, ...}
            D_len[id[0]]=0.5

    if (list[0] in Q):
        L[list[0]]= I[list[0]]

    for q_term in Q.items(): # L may not include all query terms
        if not(q_term[0] in L):
            L[q_term[0]]={} # L = {'formula':{id1:freq1, id2:freq2}, 'one': {id1:freq3}, 'right': { }}

    for list in I_D.items():
        for id in list[1].items(): # Count term occurrences in documents
            D_len[id[0]]= D_len[id[0]] + id[1] # D_len = {id1:2.5, id2:4.5, id3:7.5, ...}

    # calculate query term frequency in I_C
    CF={} # Holds query term frequency in I_C
    L_C={} # Holds inverted index from I_C for the query terms
    for list in I_C.items():
        if (list[0] in Q): # select inverted lists based on Q in I_C
            L_C[list[0]]= I_C[list[0]] # L_C = {'formula':{id1:freq1, id2:freq2}, 'one': {id1:freq3}, 'right': {id4:freq4}}
            CF[list[0]] = 0 # assign 0 to each query term # CF = {'formula':0, 'one':0, 'right':0}
    for (term, doc) in L_C.items():
        for id in doc.items():
            CF[term] = CF[term] + id[1] # CF = {'formula':0+freq1+freq2, 'one':0+freq3, 'right':0+freq4}
```

$$P(Q|D) = \prod_{i=1}^n ((1-\lambda) \frac{f_{q_i,D}}{|D|} + \lambda \frac{c_{q_i}}{|C|})$$

where c_{q_i} is the number of times query term q_i occurs in I_C , and $|C|$ is the total number of term occurrences in I_C .

Step 6:

number of time query term occurs in I_C

Task 3: Solution

$I_C = \{\text{term}: \{\text{docID1:freq1, DocID2:freq2}, \dots\}$

```
def likelihood_JM(I_C, I_D, Q, lamda): # index I_C and I_D have the form of {term:{itemId:freq}}
    L={} # Holds inverted lists of query terms
    R={} # Document scores: {docID: P(Q|D)}
    D_len={} # Document lengths: {docID: total terms}

    for list in I.items():
        for id in list[1].items():
            R[id[0]]=1 # R = {id1:1, id2:1, id3:1, ...}
            D_len[id[0]]=0.5

    if (list[0] in Q):
        L[list[0]]= I[list[0]]

    for q_term in Q.items(): # L may not include all query terms
        if not(q_term[0] in L):
            L[q_term[0]]={} # L = {'formula':{'id1:freq1, id2:freq2}, 'one': {id1:freq3}, 'right': { }}

    for list in I_D.items():
        for id in list[1].items(): # Count term occurrences in documents
            D_len[id[0]]= D_len[id[0]] + id[1] # D_len = {id1:2.5, id2:4.5, id3:7.5, ...}

    # calculate query term frequency in I_C
    CF={} # Holds query term frequency in I_C
    L_C={} # Holds inverted index from I_C for the query terms
    for list in I_C.items():
        if (list[0] in Q): # select inverted lists based on Q in I_C
            L_C[list[0]]= I_C[list[0]] # L_C = {'formula':{'id1:freq1, id2:freq2}, 'one': {id1:freq3}, 'right': {id4:freq4}}
            CF[list[0]] = 0 # assign 0 to each query term # CF = {'formula':0, 'one':0, 'right':0}
    for (term, doc) in L_C.items():
        for id in doc.items():
            CF[term] = CF[term] + id[1] # CF = {'formula':0+freq1+freq2, 'one':0+freq3, 'right':0+freq4}
    C_len = 0
    for list in I_C.items():
        for id in list[1].items(): # Count term occurrences in documents
            C_len = C_len + id[1]
```

$$P(Q|D) = \prod_{i=1}^n ((1-\lambda) \frac{f_{q_i,D}}{|D|} + \lambda \frac{c_{q_i}}{|C|})$$

where c_{q_i} is the number of times query term q_i occurs in I_C , and $|C|$ is the total number of term occurrences in I_C .

Step 6:
number of time query term occurs in I_C

Step 7:
total number of term occurrences in I_C

Task 3: define Jelinek-Mercer smoothing function likelihood_JM(I_C, I_D, Q, lamda) to
 # estimate $P(Q|D)$ for all documents D in a folder indexed by I

```
def likelihood_JM(I_C, I_D, Q, lamda): # index I_C and I_D have the form of {term:{itemId:freq}}
    # calculate query term frequency in documents indexed by I_D
    L={} # L is the selected inverted list
    R={} # R is a dictionary of docId:score
    D_len={} # D_len is a dictionary of docId:length
    for list in I_D.items():
        for id in list[1].items():
            R[id[0]]=1 # get all document IDs and initialize as 1 # R = {id1:1, id2:1, id3:1, ...}
            D_len[id[0]]=0.5 # initialize a small non-zero value as it will be used as denominator
        if (list[0] in Q): # select inverted lists based on the query
            L[list[0]] = I_D[list[0]]
    for q_term in Q.items(): # L may not include all query terms
        if not(q_term[0] in L):
            L[q_term[0]]={} # L = {'formula':{id1:freq1, id2:freq2}, 'one': {id1:freq3}, 'right': {}}
    for list in I_D.items():
        for id in list[1].items(): # Count term occurrences in documents
            D_len[id[0]] = D_len[id[0]] + id[1] # D_len = {id1:2.5, id2:4.5, id3:7.5, ...}
    # calculate query term frequency in I_C
    CF={}
    L_C={}
    for list in I_C.items():
        if (list[0] in Q): # select inverted lists based on Q in I_C
            L_C[list[0]] = I_C[list[0]]
            CF[list[0]] = 0 # assign 0 to each query term
    for (term, doc) in L_C.items():
        for id in doc.items():
            CF[term] = CF[term] + id[1]
    C_len = 0
    for list in I_C.items():
        for id in list[1].items(): # Count term occurrences in documents
            C_len = C_len + id[1]
    # using the equation
    for (d, sd) in R.items():
        for (term, f) in L.items():
            if not(d in f):
                f[d]=0
            sd = sd*(((1-lamda)*f[d]/D_len[d])+(lamda*CF[term]/C_len)) # see page 12 in wk5 lecture notes
    R[d] = sd
    return R
```

$$P(Q|D) = \prod_{i=1}^n \left((1-\lambda) \frac{f_{q_i,D}}{|D|} + \lambda \frac{c_{q_i}}{|C|} \right)$$

where c_{q_i} is the number of times query term q_i occurs in I_C , and $|C|$ is the total number of term occurrences in I_C .

From task 2

```
for (d, sd) in R.items():
    for (term, f) in L.items():
        if not(d in f):
            f[d]=0
            sd = sd*(f[d]/D_len[d])
        R[d] = sd
    return R
```

Task 4: Solution → refer Jupyter notebook

Task 4: Test the defined functions using the provided set of test documents (in folder "Test_docs") and a collection of documents (in folder "collection_C"). You need to use the 'common-english-words.txt' when indexing both sets of documents into Index_Test and Index_C, respectively. You can then test functions `likelihood_IR(Index_Test, Query)` and `likelihood_JM(Index_C, Index_Test, Query, lamda)` (you can assume `lamda = 0.4`, and `Query = {'formula':1, 'one':1}, {'share':1}` or `{'leaderboard':1, 'british':1}`).

Task 4: Solution → refer Jupyter notebook

```
curr_path=os.getcwd()
stopwords_f = open('common-english-words.txt', 'r')
stop_words = stopwords_f.read().split(',')
stopwords_f.close()

C_path = curr_path+'/collection_C'
Index_C = index_docs(C_path, stop_words) #Index I_C for all terms in <text> in collection_C
os.chdir(curr_path)

Test_path = curr_path+'/Test_docs'
Index_Test = index_docs(Test_path, stop_words) #Index I_D for all terms in <text> in Test_docs
os.chdir(curr_path)

Query1 = {'formula':1, 'one':1}
Query2 = {'share':1}
Query3 = {'leaderboard':1, 'british':1}

#test the likelihood IR model
result1 = likelihood_IR(Index_Test, Query2)
x1 = sorted(result1.items(), key=lambda x: x[1],reverse=True)
print('Likelihood Query Result -----')
for (id, w) in x1:
    if w>0:
        print('Document ID: '+id + ' and relevance weight: ' + str(w))

# test the smoothing-based IR model
result2 = likelihood_JM(Index_C, Index_Test, Query2, 0.4)
x2 = sorted(result2.items(), key=lambda x: x[1],reverse=True)
print('JM Smoothing-Based Query Result -----')
for (id, w) in x2:
    if w>0:
        print('Document ID: '+id + ' and relevance weight: ' + str(w))
```

```
Likelihood Query Result -----
Document ID: 807606 and relevance weight: 0.017937219730941704
Document ID: 809481 and relevance weight: 0.011299435028248588
Document ID: 807600 and relevance weight: 0.003179650238473768
JM Smoothing-Based Query Result -----
Document ID: 807606 and relevance weight: 0.012240164350880293
Document ID: 809481 and relevance weight: 0.008257493529264423
Document ID: 807600 and relevance weight: 0.003385622655399532
Document ID: 809495 and relevance weight: 0.0014778325123152713
```